

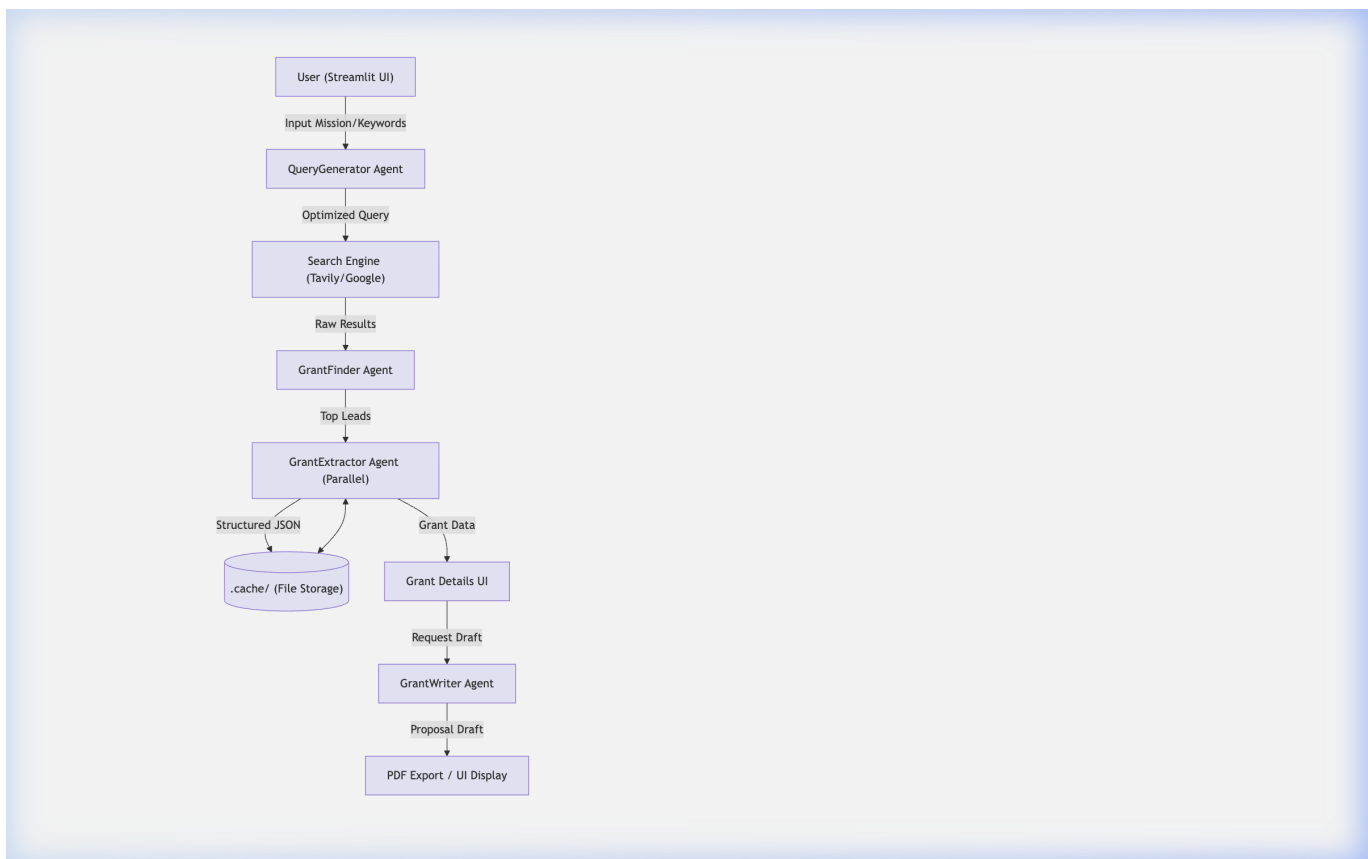
Architecture Overview

Grant Seeker AI - Architecture Overview

Grant Seeker AI is a sophisticated multi-agent system designed to automate the lifecycle of finding, analyzing, and applying for grants. It specifically targets non-profit organizations and researchers, with a focus on Canadian funding opportunities.

?? Technical Architecture

The system follows a **Hybrid Multi-Agent Architecture** built on the **Google Agent Development Kit (ADK)** and powered by **Google Gemini 1.5 Flash**. It combines sequential discovery with parallel data extraction.



?? Technology Stack

| :--- | :--- | :--- |

| **Orchestration** | [Google ADK](https://github.com/google/ai-adk) | Agent lifecycle and workflow management |

| **Frontend** | Streamlit | Multi-page interactive web application |

| **Caching** | File-based (MD5) | 24-hour TTL storage for search & extraction |

	Component	Technology	Role	
	AI Model	Gemini 1.5 Flash	Core reasoning, extraction, and drafting	
	Search API	Tavily Search	Specialized AI search and content cleaning	
	Extraction	pypdf / BeautifulSoup	Fallback mechanisms for PDF and HTML parsing	
	Validation	Pydantic	Strict schema enforcement for agent outputs	

? Key Components

1. Multi-Agent System (Backend)

The logic is encapsulated in specialized agents defined in ``backend/adk_agent.py`` and ``backend/writer_agent.py``:

- * `**`QueryGenerator`**`: Translates natural language descriptions into keyword-optimized search queries (adding Canadian context).
- * `**`GrantFinder`**`: Filters raw search results to find the 5-7 most relevant "official" grant pages.
- * `**`GrantExtractor`**`: Performs deep analysis of page content. It is "date-aware" to detect expired grants and "geography-aware" to filter for Canadian sources.
- * `**`GrantWriter`**`: A specialist agent that drafts the "Statement of Need" and "Alignment" sections of a proposal.

2. Robust Extraction Engine (``backend/content_extractor.py``)

To solve the "Untitled Grant" problem, the system uses a tiered extraction strategy:

1. `**Tavily Extract**`: Cleaned, LLM-ready text from URLs.
2. `**Google Scraper**`: Fallback scraping logic.
3. `**pypdf**`: Direct text extraction if the target is a PDF.
4. `**BeautifulSoup**`: Direct HTML scraping as a last resort (sanitizing noise tags).

3. Iterative Search & Thoroughness

The frontend allows users to set a "Minimum Results" target. The backend will iteratively refine searches and broadens queries until the desired number of relevant grants is found.

4. Smart Caching System

All search results and extracted grant data are cached in a hidden ``.cache/`` directory using MD5 hashes of queries/URLs. This ensures:

- * `**Cost Efficiency**`: Reduces redundant Gemini and Tavily API calls.
- * `**Performance**`: Repeated queries return in ~7s instead of ~30s.

? Repository Structure

```
``text
??? backend/
?   ??? adk_agent.py      # Core orchestrator and Agent definitions
?   ??? content_extractor.py # Multi-strategy scraping logic
```

```
? ??? tavily_client.py    # Tavily API wrapper with retry logic
? ??? writer_agent.py     # Specialist proposal drafting agent
??? frontend/
? ??? search_grants.py    # Main search UI with advanced filters
? ??? grant_details.py    # Deep dive view for specific grants
? ??? proposal_builder.py # AI drafting interface
??? home_page.py         # App entry point & Navigation
??? requirements.txt      # Python ecosystem (ADK, streamlit, pydantic, etc.)
````
```

## ? Data flow & Validation

Every agent interaction is governed by **Pydantic Models** (`GrantData`, `DiscoveredLeadsReport`). This ensures that the frontend receives predictable, type-safe data even when the underlying LLM responses vary.